



Qu'est-ce que Loom ?

Un agent IA local, multimodal et offline

Un modèle open-source qui **agit** sur ta machine avec des outils — 100 % local, sans internet.

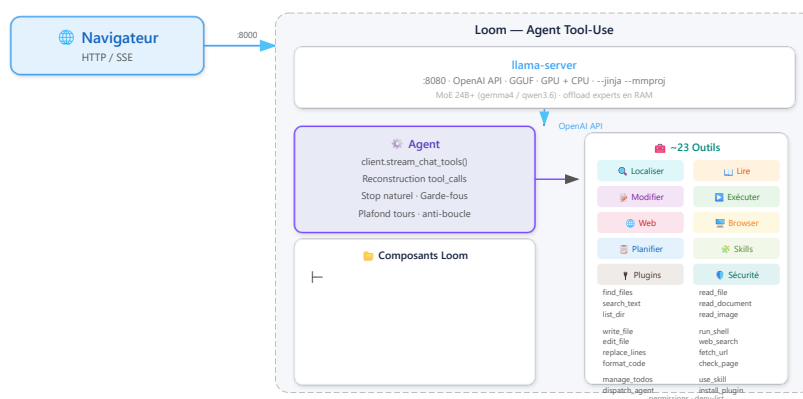
Résumé — Qu'est-ce que Loom ?

Loom est un **agent tool-use local** : une boucle qui donne à un **modèle de langue open-source** (ex. Qwen, Gemma) des **outils concrets** — fichiers, shell, web, browser — pour qu'il **localise, lise, modifie et exécute** sur de vrais projets, au lieu de se contenter de discuter.

- **Architecture** : un **harness minimal** — pas de pipeline rigide ni de mode « build » séparé, juste le modèle qui décide et enchaîne les appels d'outils.
- **Moteur** : **llama.cpp** via une API OpenAI-compatible sur :8080, avec support MoE et vision.
- **UI** : un **chat web** (Flask + Preact, zéro build) avec streaming SSE et markdown rendu.
- **Sécurité** : **mode permission** (ask / allow), deny-list, anti-SSRF et **frontière de confiance** (contenu externe = données, pas instructions).
- **Écosystème** : **skills** déclenchés par le modèle, **plugins** compatibles Claude Code, ~23 outils couvrant localiser, lire, modifier, exécuter, web, browser, planification et délégation.



Architecture technique



agent/ — client, conversation, context, session, inline_image | **tools/** — capacités appelées par le modèle | **extend/** — skills (use_skill) + plugins (store CC) | **permissions.py** — deny-list · allow/ask/deny | **config.py** — config depuis loom.config.toml | **prompts/** — prompts système (politique) | **runtime/** — serve.py, swap, hardware, profils | **web/** — Flask · SSE · templates · static | **GGUF** (model.toml) modèles/



Outils principaux



Planifier & Déléguer



UI Chat

`find_files` — recherche par motif glob

`search_text` — grep regex sur contenu

`list_dir` — liste de dossier

`read_file` — texte numéroté

`read_document` — PDF / XLSX / DOCX

`read_image` — vision de capture/schéma

`write_file` / `append_file`

`edit_file` — remplacement exact

`replace_lines` / `insert_lines`

`format_code` — ruff / prettier

`run_shell` — PowerShell / bash

`web_search` / `fetch_url`

`check_page` — headless, erreurs console

`check_interactive` — clics réels + post-cond.

`manage_todos` — bloc-notes de plan

`dispatch_agent` — sous-agent isolé

`use_skill` — charge un skill du catalogue

`install_plugin` — installe un plugin CC

Sécurité

Mode permission — ask / allow / deny

Deny-list — commandes destructrices

Anti-SSRF — hôtes internes refusés

Frontière de confiance — source externe = données

Flask + Preact/htm, zéro build

Streaming SSE

Markdown rendu

Sessions (titre inféré)

Vision (screenshot / image)

Thinking toggleable

Interruption (nouveau message)

Multi-modèles (sélecteur)

Runtime

llama.cpp → `:8080`

API OpenAI-compatible

Auto-adaptatif : GPU / CPU

Offload MoE (experts RAM)

--jinja --mmproj

Modèles par dossier

llama-swap (multi-modèle)

Python 3.12+ · uv · ruff · Flask · Preact/htm · SDK openai · llama.cpp · MoE 24B+ · Stack complète